

Introduction au Langage C

TP n°2 : Groupe A

EPITA Cyber 1
2025-2026

1 Consignes

Le sujet du TP2 est composé de **7 exercices de programmation** en langage C.

Chaque exercice contient l'**objectif de la fonction** à écrire et des **exemples d'exécution à respecter**.

Pour chaque exercice, vous devez **soumettre un fichier ".c"** dans le **dépôt associé à l'exercice sur Moodle**. Il n'est pas nécessaire d'écrire de fichier d'en-tête .h.

Chaque fichier .c doit commencer par un **bloc de commentaires avec vos informations personnelles** : Nom, prénom, date, et groupe (voir exemple ci-dessous).

Veillez à **déposer la version finale de votre code**.

Consignes additionnelles :

- Le nom du fichier .c doit être **exercice[i].c**, où **[i]** est à remplacer par le numéro de l'exercice, par exemple **exercice1.c** pour l'exercice numéro 1.
- Votre programme doit compiler avec la commande :
`$gcc -std=c11 -Wall -Wextra -Werror exercice1.c -o exercice1`
- Respectez les prototypes des fonctions données dans l'énoncé.
- Les bibliothèques autorisées sont : `stdio.h`.

Exemple : Un fichier contenant un bloc d'informations personnelles, affichant du texte dans le terminal, et permettant d'entrer du texte dans le terminal :

```
1  /*  Nom: Barrot Nathanael
2      Date: 06/10
3      Groupe: A
4  */
5
6  #include <stdio.h>                // import de stdio.h
7
8  int main(void)
9  {
10     int num=0;                    // definition de num pour
    stockage
11     printf("Entrer un entier: "); // affichage dans le terminal
12     scanf("%d", &num);            // "%d" pour entrer un entier
13     printf("L'entier est %d. \n", num); // affichage de num
14
15     return 0;
16 }
```

2 Exercice n°1 : Opérateurs arithmétiques

Objectif :

Ecrire une fonction **main** qui demande à l'utilisateur d'entrer deux entiers, et **affiche dans le terminal** la somme et le produit des deux entiers.

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur, c'est-à-dire les entrées qui ne sont pas des entiers.

Exemples :

```
1 prompt$exercice1
2 Entrer un entier: 45
3 Entrer un entier: -10
4 La somme est 35.
5 Le produit est -450.
6
7 Entrer un entier: 5
8 Entrer un entier: 0
9 La somme est 5.
10 Le produit est 0.
```

3 Exercice n°2 : Fonction min

Objectif :

Ecrire une fonction **min** qui prend en argument trois entiers, et **retourne** l'élément le plus petit.

Le prototype de la fonction doit être :

```
1 int min(int a, int b, int c);
```

Exemples d'appels :

- min(12, 16, 7) retourne 7.
- min(-6, 10, 0) retourne -6.
- min(2, 8, 2) retourne 2.
- min(7, 7, 7) retourne 7.

Conditions :

Les arguments de la fonction **min** sont des entiers relatifs.

Testez votre fonction dans votre **main** !

4 Exercice n°3 : Print Numbers

Objectif :

Ecrire une fonction **main** qui demande à l'utilisateur d'entrer un entier n , et **affiche dans le terminal** les entiers positifs, inférieurs à n , et divisible par 7.

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur, c'est-à-dire les entrées qui ne sont pas des entiers positifs.

Exemples :

```
1 prompt$exercice3
2 Entrer un entier: 14
3 7
4 14
5
6 prompt$exercice3
7 Entrer un entier: 6
8
9 prompt$exercice3
10 Entrer un entier: 32
11 7
12 14
13 21
14 28
```

5 Exercice n°4 : Fonction evenProd

Objectif :

Ecrire une fonction **evenProd** qui prend en argument un entier n , et **retourne** le produit des entiers pairs et inférieurs à n .

Le prototype de la fonction doit être :

```
1 int evenProd(int n);
```

Exemples d'appels :

- `evenProd(6)` retourne 48 (car $2*4*6 = 48$).
- `evenProd(7)` retourne 48.
- `evenProd(1)` retourne 1.
- `evenProd(0)` retourne 0.

Conditions :

L'argument de la fonction **evenSum** est un entier positif.

Il n'est pas nécessaire de gérer les arguments erronés, c'est-à-dire les arguments ne sont pas des entiers positifs.

Testez votre fonction dans votre **main** !

6 Exercice n°5 : Fonction sumDigits

Objectif :

Ecrire une fonction **sumDigits** qui prend en argument un entier n , et **retourne** la somme des chiffres qui composent n .

Le prototype de la fonction doit être :

```
1 int sumDigits(int n);
```

Exemples d'appels :

- `sumDigits(12)` retourne 3, car $1+2=3$.
- `sumDigits(174)` retourne 12, car $1+7+4=12$.
- `sumDigits(31200)` retourne 6, car $3+1+2+0+0=6$.
- `sumDigits(8)` retourne 8.

Conditions :

L'argument de la fonction **sumDigits** est un entier positif.

Il n'est pas nécessaire de gérer les arguments erronés, c'est-à-dire les arguments ne sont pas des entiers positifs.

Testez votre fonction dans votre **main** !

7 Exercice n°6 : Fonction oddCount

Objectif :

Ecrire une fonction **oddCount** qui prend en argument un entier *n*, et **retourne** le nombre de chiffres impairs parmi les chiffres qui composent *n*.

Le prototype de la fonction doit être :

```
1 int evenCount(int n);
```

Exemples d'appels :

- `evenCount(12)` retourne 1, car 1 est impair.
- `evenCount(1724)` retourne 2, car 1 et 7 sont impairs.
- `evenCount(36200)` retourne 0.
- `evenCount(777)` retourne 3.

Conditions :

L'argument de la fonction **evenCount** est un entier positif.

Il n'est pas nécessaire de gérer les arguments erronés, c'est-à-dire les arguments ne sont pas des entiers positifs.

Testez votre fonction dans votre **main** !

8 Exercice n°7 : Fonction pyramid

Objectif :

Ecrire une fonction **pyramid** qui prend en argument un entier impair n , et **affiche dans le terminal** une pyramide, composée par le symbole '*', dont la base est de taille n , et alignée à gauche.

Le prototype de la fonction doit être :

```
1 void pyramid(int n);
```

Exemples d'appels :

- `pyramid(3)` affiche dans le terminal :

```
1 *
2 ***
```

- `pyramid(7)` affiche dans le terminal :

```
1      *
2     ***
3    *****
4   *****
```

- `pyramid(13)` affiche dans le terminal :

```
1          *
2         ***
3        *****
4       *****
5      *****
6     *****
7    *****
```

Conditions :

L'argument de la fonction **pyramid** est un entier positif impair.

Il n'est pas nécessaire de gérer les arguments erronés, c'est-à-dire les arguments qui ne sont pas des entiers positifs impairs.

Testez votre fonction dans votre **main** !